

```

// For managing our event notifier.
winrt::event_token m_deviceReplacedEventToken;

// We may detect device loss on BeginDraw calls. This helper handles
this condition or other
// errors.
bool CheckForDeviceRemoved(HRESULT hr)
{
    if (hr == S_OK)
    {
        // Everything is fine: go ahead and draw.
        return true;
    }

    if (hr == DXGI_ERROR_DEVICE_REMOVED)
    {
        // We can't draw at this time, but this failure is recoverable.
        Just skip drawing for
        // now. We will be asked to draw again once the Direct3D device
        is recreated.
        return false;
    }

    // Any other error is unexpected and, therefore, fatal.
    winrt::check_hresult(hr);
    return true;
}

// Renders the text into our composition surface
void DrawText()
{
    // Begin our update of the surface pixels. If this is our first
    update, we are required
    // to specify the entire surface, which nullptr is shorthand for
    (but, as it works out,
    // any time we make an update we touch the entire surface, so we
    always pass nullptr).
    winrt::com_ptr<::ID2D1DeviceContext> d2dDeviceContext;
    POINT offset;
    if (CheckForDeviceRemoved(m_drawingSurfaceInterop-
>BeginDraw(nullptr,
    __uuidof(ID2D1DeviceContext), d2dDeviceContext.put_void(),
    &offset)))
    {
        d2dDeviceContext->Clear(D2D1::ColorF(D2D1::ColorF::Black, 0.f));

        // Create a solid color brush for the text. A more sophisticated
        application might want
        // to cache and reuse a brush across all text elements instead,
        taking care to recreate
        // it in the event of device removed.
        winrt::com_ptr<::ID2D1SolidColorBrush> brush;
        winrt::check_hresult(d2dDeviceContext->CreateSolidColorBrush(
            D2D1::ColorF(D2D1::ColorF::Black, 1.0f), brush.put()));
    }
}

```

```

        // Draw the line of text at the specified offset, which
        corresponds to the top-left
        // corner of our drawing surface. Notice we don't call BeginDraw
        on the D2D device
        // context; this has already been done for us by the composition
        API.
        d2dDeviceContext->DrawTextLayout(D2D1::Point2F((float)offset.x,
        (float)offset.y), m_text.get(),
        brush.get());

        // Our update is done. EndDraw never indicates rendering device
        removed, so any
        // failure here is unexpected and, therefore, fatal.
        winrt::check_hresult(m_drawingSurfaceInterop->EndDraw());
    }
}
};

struct DeviceLostEventArgs
{
    DeviceLostEventArgs(IDirect3DDevice const& device) : m_device(device) {}
    IDirect3DDevice Device() { return m_device; }
    static DeviceLostEventArgs Create(IDirect3DDevice const& device) {
return DeviceLostEventArgs{ device }; }

private:
    IDirect3DDevice m_device;
};

struct DeviceLostHelper
{
    DeviceLostHelper() = default;

    ~DeviceLostHelper()
    {
        StopWatchingCurrentDevice();
        m_onDeviceLostHandler = nullptr;
    }

    IDirect3DDevice CurrentlyWatchedDevice() { return m_device; }

    void WatchDevice(winrt::com_ptr<::IDXGIDevice> const& dxgiDevice)
    {
        // If we're currently listening to a device, then stop.
        StopWatchingCurrentDevice();

        // Set the current device to the new device.
        m_device = nullptr;

        winrt::check_hresult(::CreateDirect3D11DeviceFromDXGIDevice(dxgiDevice.get()
        , reinterpret_cast<::IInspectable*>(winrt::put_abi(m_device))));

        // Get the DXGI Device.
        m_dxgiDevice = dxgiDevice;
    }
};

```

```

        // QI For the ID3D11Device4 interface.
        winrt::com_ptr<::ID3D11Device4> d3dDevice{
m_dxgiDevice.as<::ID3D11Device4>() };

        // Create a wait struct.
        m_onDeviceLostHandler = nullptr;
        m_onDeviceLostHandler =
::CreateThreadpoolWait(DeviceLostHelper::OnDeviceLost, (PVOID)this,
nullptr);

        // Create a handle and a cookie.
        m_eventHandle.attach(::CreateEvent(nullptr, false, false, nullptr));
        winrt::check_bool(bool{ m_eventHandle });
        m_cookie = 0;

        // Register for device lost.
        ::SetThreadpoolWait(m_onDeviceLostHandler, m_eventHandle.get(),
nullptr);
        winrt::check_hresult(d3dDevice->RegisterDeviceRemovedEvent(m_eventHandle.get(), &m_cookie));
    }

    void StopWatchingCurrentDevice()
    {
        if (m_dxgiDevice)
        {
            // QI For the ID3D11Device4 interface.
            auto d3dDevice{ m_dxgiDevice.as<::ID3D11Device4>() };

            // Unregister from the device lost event.
            ::CloseThreadpoolWait(m_onDeviceLostHandler);
            d3dDevice->UnregisterDeviceRemoved(m_cookie);

            // Clear member variables.
            m_onDeviceLostHandler = nullptr;
            m_eventHandle.close();
            m_cookie = 0;
            m_device = nullptr;
        }
    }

    void DeviceLost(winrt::delegate<DeviceLostHelper const*,
DeviceLostEventArgs const&> const& handler)
    {
        m_deviceLost = handler;
    }

    winrt::delegate<DeviceLostHelper const*, DeviceLostEventArgs const&>
m_deviceLost;

private:
    void RaiseDeviceLostEvent(IDirect3DDevice const& oldDevice)
    {
        m_deviceLost(this, DeviceLostEventArgs::Create(oldDevice));
    }

```

```

        static void CALLBACK OnDeviceLost(PTP_CALLBACK_INSTANCE /* instance */,
        PVOID context, PTP_WAIT /* wait */, TP_WAIT_RESULT /* waitResult */)
        {
            auto deviceLostHelper = reinterpret_cast<DeviceLostHelper*>
            (context);
            auto oldDevice = deviceLostHelper->m_device;
            deviceLostHelper->StopWatchingCurrentDevice();
            deviceLostHelper->RaiseDeviceLostEvent(oldDevice);
        }

private:
    IDirect3DDevice m_device;
    winrt::com_ptr<::IDXGIDevice> m_dxgiDevice;
    PTP_WAIT m_onDeviceLostHandler{ nullptr };
    winrt::handle m_eventHandle;
    DWORD m_cookie{ 0 };
};

struct SampleApp : implements<SampleApp, IFrameworkViewSource,
IFrameworkView>
{
    IFrameworkView CreateView()
    {
        return *this;
    }

    void Initialize(CoreApplicationView const&)
    {
        // Run once when the application starts up
        void Initialize()
        {
            // Create a Direct2D device.
            CreateDirect2DDevice();

            // To create a composition graphics device, we need to QI for
            another interface
            winrt::com_ptr<abi::ICompositorInterop> compositorInterop{
            m_compositor.as<abi::ICompositorInterop>() };

            // Create a graphics device backed by our D3D device
            winrt::com_ptr<abi::ICompositionGraphicsDevice>
            compositionGraphicsDeviceIface;
            winrt::check_hresult(compositorInterop->CreateGraphicsDevice(
                m_d2dDevice.get(),
                compositionGraphicsDeviceIface.put()));
            m_compositionGraphicsDevice =
            compositionGraphicsDeviceIface.as<CompositionGraphicsDevice>();
        }

        void Load(hstring const&)
        {
        }
    }
};

```

```

void Uninitialize()
{
}

void Run()
{
    CoreWindow window = CoreWindow::GetForCurrentThread();
    window.Activate();

    CoreDispatcher dispatcher = window.Dispatcher();
    dispatcher.ProcessEvents(CoreProcessEventsOption::ProcessUntilQuit);
}

void SetWindow(CoreWindow const& window)
{
    m_compositor = Compositor{};
    m_target = m_compositor.CreateTargetForCurrentView();
    ContainerVisual root = m_compositor.CreateContainerVisual();
    m_target.Root(root);

    Initialize();

    winrt::check_hresult(
        ::DWriteCreateFactory(
            DWRITE_FACTORY_TYPE_SHARED,
            __uuidof(m_dWriteFactory),
            reinterpret_cast<::IUnknown**>(m_dWriteFactory.put())
        )
    );

    winrt::check_hresult(
        m_dWriteFactory->CreateTextFormat(
            L"Segoe UI",
            nullptr,
            DWRITE_FONT_WEIGHT_REGULAR,
            DWRITE_FONT_STYLE_NORMAL,
            DWRITE_FONT_STRETCH_NORMAL,
            36.f,
            L"en-US",
            m_textFormat.put()
        )
    );

    Rect windowBounds{ window.Bounds() };
    std::wstring text{ L"Hello, World!" };

    winrt::check_hresult(
        m_dWriteFactory->CreateTextLayout(
            text.c_str(),
            (uint32_t)text.size(),
            m_textFormat.get(),
            windowBounds.Width,
            windowBounds.Height,
            m_textLayout.put()
        )
    );
}

```

```

    )
};

Visual textVisual{ CreateVisualFromTextLayout(m_textLayout) };
textVisual.Size({ windowBounds.Width, windowBounds.Height });
root.Children().InsertAtTop(textVisual);
}

// Called when Direct3D signals the device lost event.
void OnDirect3DDeviceLost(DeviceLostHelper const* /* sender */,
DeviceLostEventArgs const& /* args */)
{
    // Create a new Direct2D device.
    CreateDirect2DDevice();

    // Restore our composition graphics device to good health.
    winrt::com_ptr<abi::ICompositionGraphicsDeviceInterop>
compositionGraphicsDeviceInterop{
m_compositionGraphicsDevice.as<abi::ICompositionGraphicsDeviceInterop>() };
    winrt::check_hresult(compositionGraphicsDeviceInterop-
>SetRenderingDevice(m_d2dDevice.get()));
}

// Create a surface that is asynchronously filled with an image
ICompositionSurface
CreateSurfaceFromTextLayout(winrt::com_ptr<::IDWriteTextLayout> const& text)
{
    // Create our wrapper object that will handle downloading and
    decoding the image (assume
    // throwing new here).
    SampleText textSurface{ text, m_compositionGraphicsDevice };

    // The caller is only interested in the underlying surface.
    return textSurface.Surface();
}

// Create a visual that holds an image.
Visual CreateVisualFromTextLayout(winrt::com_ptr<::IDWriteTextLayout>
const& text)
{
    // Create a sprite visual
    SpriteVisual spriteVisual{ m_compositor.CreateSpriteVisual() };

    // The sprite visual needs a brush to hold the image.
    CompositionSurfaceBrush surfaceBrush{
m_compositor.CreateSurfaceBrush(CreateSurfaceFromTextLayout(text))
};

    // Associate the brush with the visual.
    CompositionBrush brush{ surfaceBrush.as<CompositionBrush>() };
    spriteVisual.Brush(brush);

    // Return the visual to the caller as an IVisual.
    return spriteVisual;
}

```

```

    }

private:
    CompositionTarget m_target{ nullptr };
    Compositor m_compositor{ nullptr };
    winrt::com_ptr<::ID2D1Device> m_d2dDevice;
    winrt::com_ptr<::IDXGIDevice> m_dxgiDevice;
    //winrt::com_ptr<abi::ICompositionGraphicsDevice>
m_compositionGraphicsDevice;
    CompositionGraphicsDevice m_compositionGraphicsDevice{ nullptr };
    std::vector<SampleText> m_textSurfaces;
    DeviceLostHelper m_deviceLostHelper;
    winrt::com_ptr<::IDWriteFactory> m_dWriteFactory;
    winrt::com_ptr<::IDWriteTextFormat> m_textFormat;
    winrt::com_ptr<::IDWriteTextLayout> m_textLayout;

    // This helper creates a Direct2D device, and registers for a device
loss
    // notification on the underlying Direct3D device. When that
notification is
    // raised, the OnDirect3DDeviceLost method is called.
    void CreateDirect2DDevice()
    {
        uint32_t createDeviceFlags = D3D11_CREATE_DEVICE_BGRA_SUPPORT;

        // Array with DirectX hardware feature levels in order of
preference.
        D3D_FEATURE_LEVEL featureLevels[] =
        {
            D3D_FEATURE_LEVEL_11_1,
            D3D_FEATURE_LEVEL_11_0,
            D3D_FEATURE_LEVEL_10_1,
            D3D_FEATURE_LEVEL_10_0,
            D3D_FEATURE_LEVEL_9_3,
            D3D_FEATURE_LEVEL_9_2,
            D3D_FEATURE_LEVEL_9_1
        };

        // Create the Direct3D 11 API device object and a corresponding
context.
        winrt::com_ptr<::ID3D11Device> d3DDevice;
        winrt::com_ptr<::ID3D11DeviceContext> d3DImmediateContext;
        D3D_FEATURE_LEVEL d3dFeatureLevel{ D3D_FEATURE_LEVEL_9_1 };

        winrt::check_hresult(
            ::D3D11CreateDevice(
                nullptr, // Default adapter.
                D3D_DRIVER_TYPE_HARDWARE,
                0, // Not asking for a software driver, so not passing a
module to one.
                createDeviceFlags, // Set debug and Direct2D compatibility
flags.
                featureLevels,
                ARRAYSIZE(featureLevels),

```

```

        D3D11_SDK_VERSION,
        d3DDevice.put(),
        &d3dFeatureLevel,
        d3DImmediateContext.put()
    )
};

// Initialize Direct2D resources.
D2D1_FACTORY_OPTIONS d2d1FactoryOptions{ D2D1_DEBUG_LEVEL_NONE };

// Initialize the Direct2D Factory.
winrt::com_ptr<::ID2D1Factory1> d2D1Factory;
winrt::check_hresult(
    ::D2D1CreateFactory(
        D2D1_FACTORY_TYPE_SINGLE_THREADED,
        __uuidof(d2D1Factory),
        &d2d1FactoryOptions,
        d2D1Factory.put_void()
    )
);

// Create the Direct2D device object.
// Obtain the underlying DXGI device of the Direct3D device.
m_dxgiDevice = d3DDevice.as<::IDXGIDevice>();

m_d2dDevice = nullptr;
winrt::check_hresult(
    d2D1Factory->CreateDevice(m_dxgiDevice.get(), m_d2dDevice.put())
);

m_deviceLostHelper.WatchDevice(m_dxgiDevice);
m_deviceLostHelper.DeviceLost({ this,
&SampleApp::OnDirect3DDeviceLost });
};

int __stdcall wWinMain(HINSTANCE, HINSTANCE, PWSTR, int)
{
    CoreApplication::Run(winrt::make<SampleApp>());
}

```

C++/CX usage example

⚠ Note

This code example exists to help you maintain your C++/CX application. But we recommend that you use **C++/WinRT** for new applications. C++/WinRT is an entirely standard modern C++17 language projection for Windows Runtime

(WinRT) APIs, implemented as a header-file-based library, and designed to provide you with first-class access to the modern Windows API.

The C++/CX code example below omits the DirectWrite and Direct2D parts of the example.

C++/CX

```
//-----  
----  
//  
// Copyright (C) Microsoft. All rights reserved.  
//  
//-----  
----  
  
#include "stdafx.h"  
  
using namespace Microsoft::WRL;  
using namespace Windows::Foundation;  
using namespace Windows::Graphics::DirectX;  
using namespace Windows::UI::Composition;  
  
// This is an app-provided helper to render lines of text  
class SampleText  
{  
private:  
    // The text to draw  
    ComPtr<IDWriteTextLayout> _text;  
  
    // The composition surface that we use in the visual tree  
    ComPtr<ICompositionDrawingSurfaceInterop> _drawingSurfaceInterop;  
  
    // The device that owns the surface  
    ComPtr<ICompositionGraphicsDevice> _compositionGraphicsDevice;  
  
    // For managing our event notifier  
    EventRegistrationToken _deviceReplacedEventToken;  
  
public:  
    SampleText(IDWriteTextLayout* text, ICompositionGraphicsDevice*  
compositionGraphicsDevice) :  
        _text(text),  
        _compositionGraphicsDevice(compositionGraphicsDevice)  
    {  
        // Create the surface just big enough to hold the formatted text  
        block.  
        DWRITE_TEXT_METRICS metrics;  
        FailFastOnFailure(text->GetMetrics(&metrics));  
        Windows::Foundation::Size surfaceSize = { metrics.width,  
metrics.height };  
        ComPtr<ICompositionDrawingSurface> drawingSurface;
```

```

        FailFastOnFailure(_compositionGraphicsDevice->CreateDrawingSurface(
            surfaceSize,
            DirectXPixelFormat::DirectXPixelFormat_B8G8R8A8UIntNormalized,
            DirectXAlphaMode::DirectXAlphaMode_Ignore,
            &drawingSurface));

    // Cache the interop pointer, since that's what we always use.
    FailFastOnFailure(drawingSurface.As(&_drawingSurfaceInterop));

    // Draw the text
    DrawText();

    // If the rendering device is lost, the application will recreate
    and replace it. We then
    // own redrawing our pixels.
    FailFastOnFailure(_compositionGraphicsDevice-
>add_RenderingDeviceReplaced(
        Callback<RenderingDeviceReplacedEventHandler>([this](
            ICompositionGraphicsDevice* source,
            IRenderingDeviceReplacedEventArgs* args)
            -> HRESULT
        {
            // Draw the text again
            DrawText();
            return S_OK;
        }).Get(),
        &_deviceReplacedEventToken));
    }

    ~SampleText()
    {
        FailFastOnFailure(_compositionGraphicsDevice-
>remove_RenderingDeviceReplaced(
            _deviceReplacedEventToken));
    }

    // Return the underlying surface to the caller
    ComPtr<ICompositionSurface> get_Surface()
    {
        // To the caller, the fact that we have a drawing surface is an
        implementation detail.
        // Return the base interface instead
        ComPtr<ICompositionSurface> surface;
        FailFastOnFailure(_drawingSurfaceInterop.As(&surface));
        return surface;
    }

private:
    // We may detect device loss on BeginDraw calls. This helper handles
    this condition or other
    // errors.
    bool CheckForDeviceRemoved(HRESULT hr)
    {
        if (SUCCEEDED(hr))
        {

```

```

        // Everything is fine -- go ahead and draw
        return true;
    }
    else if (hr == DXGI_ERROR_DEVICE_REMOVED)
    {
        // We can't draw at this time, but this failure is recoverable.
        Just skip drawing for
        // now. We will be asked to draw again once the Direct3D device
        is recreated
        return false;
    }
    else
    {
        // Any other error is unexpected and, therefore, fatal
        FailFast();
    }
}

// Renders the text into our composition surface
void DrawText()
{
    // Begin our update of the surface pixels. If this is our first
    update, we are required
    // to specify the entire surface, which nullptr is shorthand for
    (but, as it works out,
    // any time we make an update we touch the entire surface, so we
    always pass nullptr).
    ComPtr<ID2D1DeviceContext> d2dDeviceContext;
    POINT offset;
    if (CheckForDeviceRemoved(_drawingSurfaceInterop->BeginDraw(nullptr,
        __uuidof(ID2D1DeviceContext), &d2dDeviceContext, &offset)))
    {
        // Create a solid color brush for the text. A more sophisticated
        application might want
        // to cache and reuse a brush across all text elements instead,
        taking care to recreate
        // it in the event of device removed.
        ComPtr<ID2D1SolidColorBrush> brush;
        FailFastOnFailure(d2dDeviceContext->CreateSolidColorBrush(
            D2D1::ColorF(D2D1::ColorF::Black, 1.0f), &brush));

        // Draw the line of text at the specified offset, which
        corresponds to the top-left
        // corner of our drawing surface. Notice we don't call BeginDraw
        on the D2D device
        // context; this has already been done for us by the composition
        API.
        d2dDeviceContext->DrawTextLayout(D2D1::Point2F(offset.x,
        offset.y), _text.Get(),
            brush.Get());

        // Our update is done. EndDraw never indicates rendering device
        removed, so any
        // failure here is unexpected and, therefore, fatal.
        FailFastOnFailure(_drawingSurfaceInterop->EndDraw());
    }
}

```

```

    }
}
};

class SampleApp
{
    ComPtr<ICompositor> _compositor;
    ComPtr<ID2D1Device> _d2dDevice;
    ComPtr<ICompositionGraphicsDevice> _compositionGraphicsDevice;
    std::vector<ComPtr<SampleText>> _textSurfaces;

public:
    // Run once when the application starts up
    void Initialize(ICompositor* compositor)
    {
        // Cache the compositor (created outside of this method)
        _compositor = compositor;

        // Create a Direct2D device (helper implementation not shown here)
        FailFastOnFailure(CreateDirect2DDevice(&_d2dDevice));

        // To create a composition graphics device, we need to QI for
another interface
        ComPtr<ICompositorInterop> compositorInterop;
        FailFastOnFailure(_compositor.As(&compositorInterop));

        // Create a graphics device backed by our D3D device
        FailFastOnFailure(compositorInterop->CreateGraphicsDevice(
            _d2dDevice.Get(),
            &_compositionGraphicsDevice));
    }

    // Called when Direct3D signals the device lost event
    void OnDirect3DDeviceLost()
    {
        // Create a new device

        FailFastOnFailure(CreateDirect2DDevice(_d2dDevice.ReleaseAndGetAddressOf()))
        ;

        // Restore our composition graphics device to good health
        ComPtr<ICompositionGraphicsDeviceInterop>
compositionGraphicsDeviceInterop;

        FailFastOnFailure(_compositionGraphicsDevice.As(&compositionGraphicsDeviceIn
terop));
        FailFastOnFailure(compositionGraphicsDeviceInterop-
>SetRenderingDevice(_d2dDevice.Get()));
    }

    // Create a surface that is asynchronously filled with an image
    ComPtr<ICompositionSurface>
CreateSurfaceFromTextLayout(IDWriteTextLayout* text)
    {
        // Create our wrapper object that will handle downloading and

```

```

decoding the image (assume
    // throwing new here)
    SampleText* textSurface = new SampleText(text,
_compositionGraphicsDevice.Get());

    // Keep our image alive
    _textSurfaces.push_back(textSurface);

    // The caller is only interested in the underlying surface
    return textSurface->get_Surface();
}

// Create a visual that holds an image
ComPtr<IVisual> CreateVisualFromTextLayout(IDWriteTextLayout* text)
{
    // Create a sprite visual
    ComPtr<ISpriteVisual> spriteVisual;
    FailFastOnFailure(_compositor->CreateSpriteVisual(&spriteVisual));

    // The sprite visual needs a brush to hold the image
    ComPtr<ICompositionSurfaceBrush> surfaceBrush;
    FailFastOnFailure(_compositor->CreateSurfaceBrushWithSurface(
        CreateSurfaceFromTextLayout(text).Get(),
        &surfaceBrush));

    // Associate the brush with the visual
    ComPtr<ICompositionBrush> brush;
    FailFastOnFailure(surfaceBrush.As(&brush));
    FailFastOnFailure(spriteVisual->put_Brush(brush.Get()));

    // Return the visual to the caller as the base class
    ComPtr<IVisual> visual;
    FailFastOnFailure(spriteVisual.As(&visual));

    return visual;
}

private:
    // This helper (implementation not shown here) creates a Direct2D device
    and registers
    // for a device loss notification on the underlying Direct3D device.
    When that notification is
    // raised, assume the OnDirect3DDeviceLost method is called.
    HRESULT CreateDirect2DDevice(ID2D1Device** ppDevice);
};

```

Tailoring effects & experiences using Windows UI

Article • 03/16/2023

Windows UI provides many beautiful effects, animations, and means for differentiation. However, meeting user expectations for performance and customizability is still a necessary part of creating successful applications. The Universal Windows Platform supports a large, diverse family of devices, which have different features and capabilities. In order to provide an inclusive experience for all your users, you need to ensure your applications scale across devices and respect user preferences. UI tailoring can provide an efficient way to leverage a device's capabilities and ensure a pleasant and inclusive user experience.

UI tailoring is a broad category encompassing work for performant, beautiful UI with respect to the following areas:

- Respecting and adapting to user settings for effects
- Accommodating user settings for animations
- Optimizing UI for the given hardware capabilities

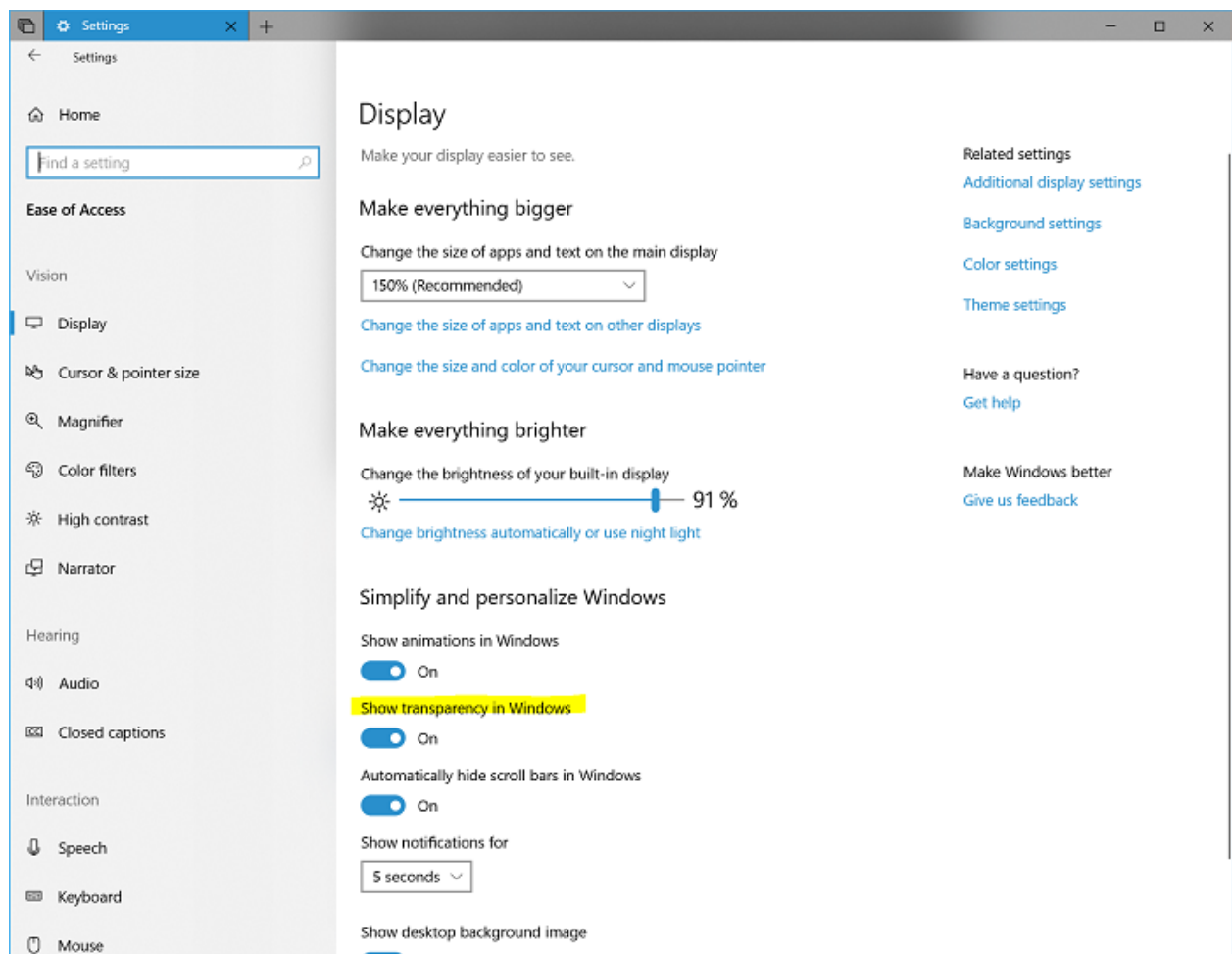
Here, we'll cover how to tailor your effects and animations with the Visual Layer in the areas above, but there are many other means to tailor your application to ensure a great end user experience. Guidance docs are available on how to [tailor your UI](#) for various devices and [create responsive UI](#).

User effects settings

Users may customize their Windows experience for a variety of reasons, which applications should respect and adapt to. One area end users can control is changing the types of effects they see used throughout their system.

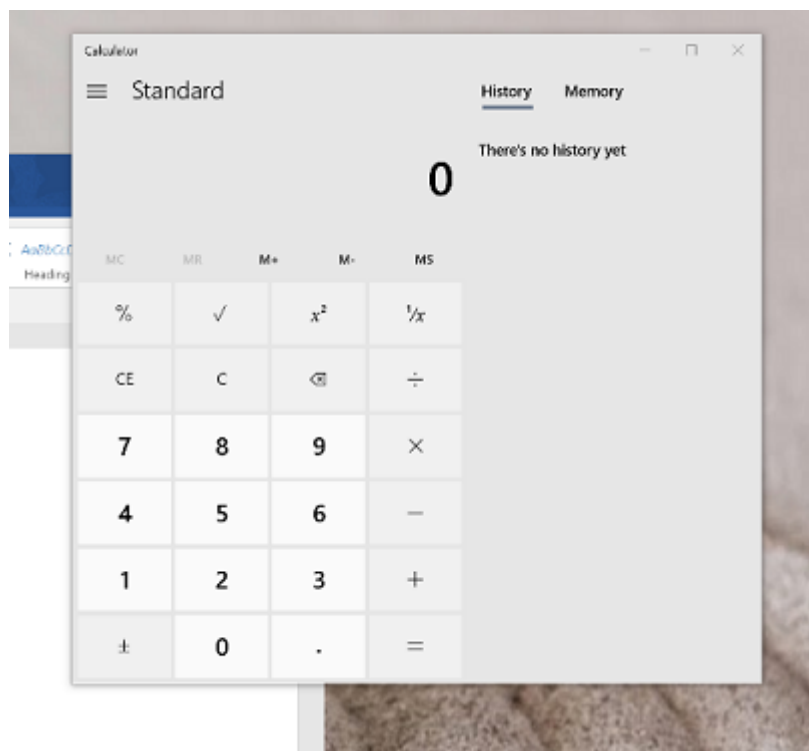
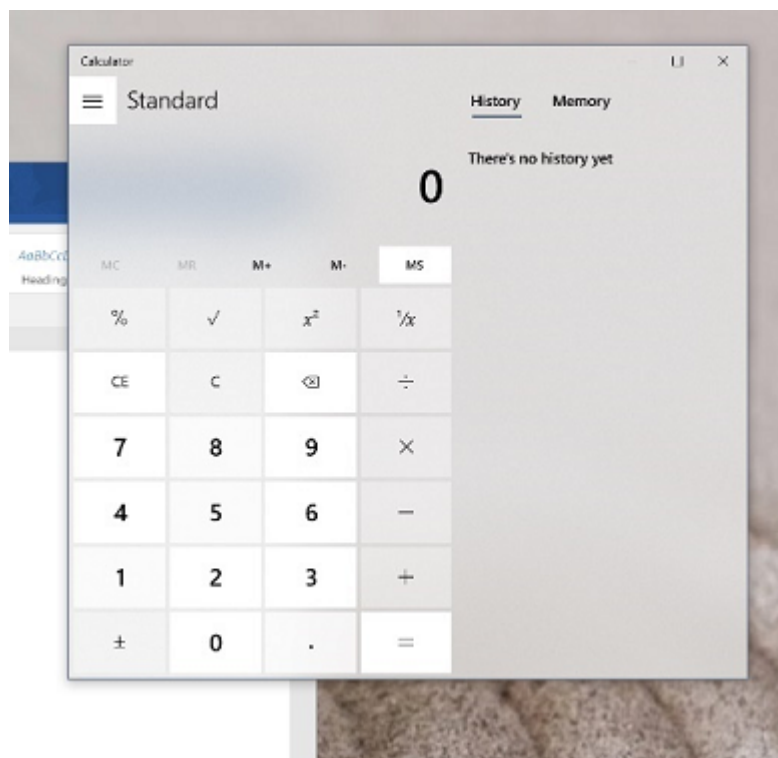
Transparency effects settings

One such effect setting users can customize is turning transparency effects on/off. This can be found in the Settings app under Personalization > Colors, or through Settings app > Ease of Access > Display.



When turned on, any effect that uses transparency will appear as expected. This applies to Acrylic, HostBackdropBrush, or any custom effect graph that is not fully opaque.

When turned off, acrylic material will automatically fall back to a solid color because XAML's acrylic brush has listened to this event by default. Here, we see the calculator app appropriately falling back to a solid color when transparency effects are not enabled:



However, for any custom effects the application needs to respond to the [UISettings.AdvancedEffectsEnabled](#) property or [AdvancedEffectsEnabledChanged](#) event and switch out the effect/effect graph to use an effect that has no transparency. An example of this is below:

C#

```
public MainPage()
{
    var uisettings = new UISettings();
    bool advancedEffects = uisettings.AdvancedEffectsEnabled;
    uisettings.AdvancedEffectsEnabledChanged +=
```



```

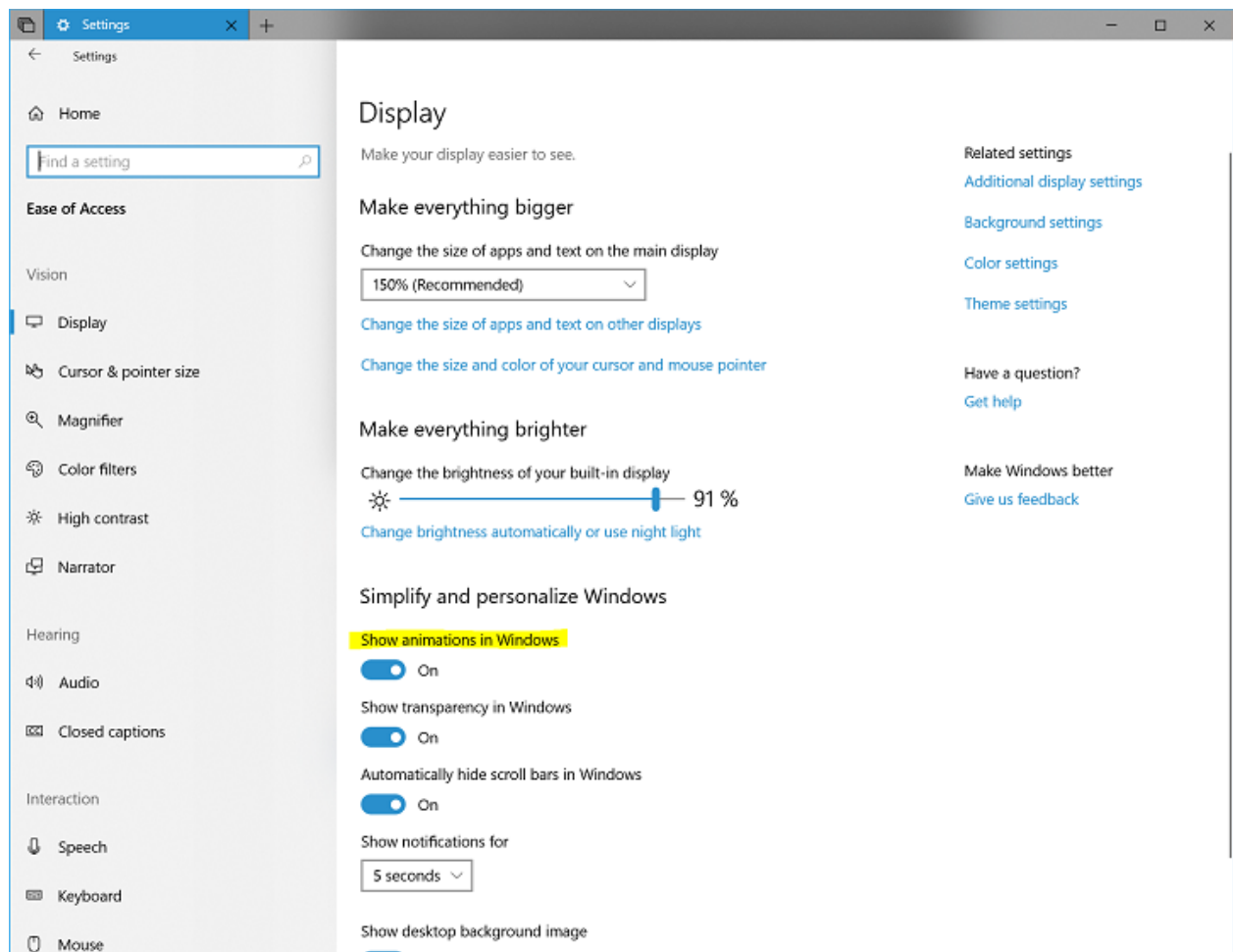
UISettings_AdvancedEffectsEnabledChanged;
}

private void UISettings_AdvancedEffectsEnabledChanged(UISettings sender,
object args)
{
    // TODO respond to sender.AdvancedEffectsEnabled
}

```

Animations settings

Similarly, applications should listen and respond to the [UISettings.AnimationsEnabled](#) property to ensure animations are turned on or off based on user settings in Settings > Ease of Access > Display.



C#

```

public MainPage()
{
    var uisettings = new UISettings();
    bool animationsEnabled = uisettings.AnimationsEnabled;
    // TODO respond to animations settings
}

```

Leveraging the capabilities API

By leveraging the [CompositionCapabilities](#) APIs, you can detect which composition features are available and performant on given hardware and tailor the design to ensure end users get a performant and beautiful experience on any device. The APIs provide a means to check hardware system capabilities in order to implement graceful effect scaling across a variety of form factors. This makes it easy to appropriately tailor the application to create a beautiful and seamless end user experience.

This API provides methods and an event listener that can be used to make effect scaling decisions for the application UI. The feature detects how well the system can handle complex composition and rendering operations and then returns the information in an easy-to-consume model for developers to utilize.

Using composition capabilities

The CompositionCapabilities functionality is already being leveraged for features like Acrylic material, where the material falls back to a more performant effect depending on the scenario and hardware.

The API can be added to existing code in a few easy steps.

1. Acquire the capabilities object in your application's constructor.

```
C#  
  
_capabilities = CompositionCapabilities.GetForCurrentView();
```

2. Register a capabilities changed event listener for your app.

```
C#  
  
_capabilities.Changed += HandleCapabilitiesChanged;
```

3. Add content to the event callback method to handle various capabilities levels. This may or may not be similar to the next step below.
 4. When using effects, check the capabilities object first. Consider using conditional checks or switch control statements, depending on how you want to tailor the effects.
-

C#

```
if (_capabilities.AreEffectsSupported())
{
    // Add incremental effects updates here

    if (_capabilities.AreEffectsFast())
    {
        // Add more advanced effects here where applicable
    }
}
```

Full example code can be found on the [Windows UI Github repo](#).

Fast vs. slow effects

Based on feedback from the provided [AreEffectsSupported](#) and [AreEffectsFast](#) methods in the CompositionCapabilities API, the application can decide to swap expensive or unsupported effects for other effects of their choice that are optimized for the device. Some effects are known to consistently be more resource intensive than others and should be used sparingly, and other effects can be used more freely. For all effects, however, care should be used when chaining and animating as some scenarios or combinations may change the performance characteristics of the effect graph. Below are some rule of thumb performance characteristics for individual effects:

- Effects that are known to have high performance impact are as follows – Gaussian Blur, Shadow Mask, BackDropBrush, HostBackDropBrush, and Layer Visual. These are not recommended for low end devices ([feature level 9.1-9.3](#)), and should be used judiciously on high end devices.
- Effects with medium performance impact include Color Matrix, certain Blend Effect BlendModes (Luminosity, Color, Saturation, and Hue), SpotLight, SceneLightingEffect, and (depending on scenario) BorderEffect. These effects may work with certain scenarios on low end devices, but care should be used when chaining and animating. Recommend restricting use to two or less and animating on transitions only.
- All other effects have low performance impact and work in all reasonable scenarios when animating and chaining.

Related articles

- [UWP Responsive Design Techniques](#)
- [UWP Device Tailoring](#)

Using the Visual Layer with XAML

Article • 03/16/2023

Most apps that consume Visual Layer capabilities will use XAML to define the main UI content. In the Windows 10 Anniversary Update, there are new features in the XAML framework and the Visual Layer that make it easier to combine these two technologies to create stunning user experiences. XAML and Visual Layer interop functionality can be used to create advanced animations and effects not available using XAML APIs alone. This includes:

- Brush effects like blur and frosted glass
- Dynamic lighting effects
- Scroll driven animations and parallax
- Automatic layout animations
- Pixel-perfect drop shadows

These effects and animations can be applied to existing XAML content, so you don't have to dramatically restructure your XAML app to take advantage of the new functionality. Layout animations, shadows, and blur effects are covered in the Recipes section below. For a code sample implementing parallax, see the [ParallaxingListItems sample](#). The [WindowsCompositionSamples repository](#) also has several other samples for implementing animations, shadows and effects.

The XamlCompositionBrushBase class

XamlCompositionBrush provides a base class for XAML brushes that paint an area with a **CompositionBrush**. This can be used to easily apply composition effects like blur or frosted glass to XAML UI elements.

See the [Brushes](#) section for more info on using brushes with XAML UI.

For code examples, see the reference page for [XamlCompositionBrushBase](#).

The XamlLight class

XamlLight provides a base class for XAML lighting effects that dynamically light an area with a **CompositionLight**.

See the [Lighting](#) section for more info on using lights, including lighting XAML UI elements.

For code examples, see the reference page for [XamlLight](#).

The ElementCompositionPreview class

[ElementCompositionPreview](#) is a static class that provides XAML and Visual Layer interop functionality. For an overview of the Visual Layer and its functionality, see [Visual Layer](#). The [ElementCompositionPreview](#) class provides the following methods:

- [GetElementVisual](#): Get a "handout" Visual that is used to render this element
- [SetElementChildVisual](#): Sets a "handin" Visual as the last child of this element's visual tree. This Visual will draw on top of the rest of the element.
- [GetElementChildVisual](#): Retrieve the Visual set using [SetElementChildVisual](#)
- [GetScrollViewerManipulationPropertySet](#): Get an object that can be used to create 60fps animations based on scroll offset in a [ScrollViewer](#)

Remarks on ElementCompositionPreview.GetElementVisual

[ElementCompositionPreview.GetElementVisual](#) returns a "handout" Visual that is used to render the given [UIElement](#). Properties such as [Visual.Opacity](#), [Visual.Offset](#), and [Visual.Size](#) are set by the XAML framework based on the state of the [UIElement](#). This enables techniques such as implicit reposition animations (see *Recipes*).

Note that since [Offset](#) and [Size](#) are set as the result of XAML framework layout, developers should be careful when modifying or animating these properties. Developers should only modify or animate [Offset](#) when the element's top-left corner has the same position as that of its parent in layout. [Size](#) should generally not be modified, but accessing the property may be useful. For example, the Drop Shadow and Frosted Glass samples below use [Size](#) of a handout Visual as input to an animation.

As an additional caveat, updated properties of the handout Visual will not be reflected in the corresponding [UIElement](#). So for example, setting [UIElement.Opacity](#) to 0.5 will set the corresponding handout Visual's [Opacity](#) to 0.5. However, setting the handout Visual's [Opacity](#) to 0.5 will cause the content to appear at 50% opacity, but will not change the value of the corresponding [UIElement](#)'s [Opacity](#) property.

Example of Offset animation

Incorrect

XAML

```
<Border>
    <Image x:Name="MyImage" Margin="5" />
</Border>
```

C#

```
// Doesn't work because Image has a margin!
ElementCompositionPreview.GetElementVisual(MyImage).StartAnimation("Offset",
parallaxAnimation);
```

Correct

XAML

```
<Border>
    <Canvas Margin="5">
        <Image x:Name="MyImage" />
    </Canvas>
</Border>
```

C#

```
// This works because the Canvas parent doesn't generate a layout offset.
ElementCompositionPreview.GetElementVisual(MyImage).StartAnimation("Offset",
parallaxAnimation);
```

The ElementCompositionPreview.SetElementChildVisual method

ElementCompositionPreview.SetElementChildVisual allows the developer to supply a “handin” Visual that will appear as part of an element’s Visual Tree. This allows developers to create a “Composition Island” where Visual-based content can appear inside a XAML UI. Developers should be conservative about using this technique because Visual-based content will not have the same accessibility and user experience guarantees of XAML content. Therefore, it is generally recommended that this technique only be used when necessary to implement custom effects such as those found in the Recipes section below.

GetAlphaMask methods

[Image](#), [TextBlock](#), and [Shape](#) each implement a method called **GetAlphaMask** that returns a **CompositionBrush** representing a grayscale image with the shape of the element. This **CompositionBrush** can serve as an input for a **Composition DropShadow**, so the shadow can reflect the shape of the element instead of a rectangle. This enables pixel perfect, contour-based shadows for text, images with alpha, and shapes. See *Drop Shadow* below for an example of this API.

Recipes

Reposition animation

Using **Composition Implicit Animations**, a developer can automatically animate changes in an element's layout relative to its parent. For example, if you change the **Margin** of the button below, it will automatically animate to its new layout position.

Implementation overview

1. Get the handout **Visual** for the target element
2. Create an **ImplicitAnimationCollection** that automatically animates changes in the **Offset** property
3. Associate the **ImplicitAnimationCollection** with the backing **Visual**

XAML

```
<Button x:Name="RepositionTarget" Content="Click Me" />
```

C#

```
public MainPage()
{
    InitializeComponent();
    InitializeRepositionAnimation(RepositionTarget);
}

private void InitializeRepositionAnimation(UIElement repositionTarget)
{
    var targetVisual =
        ElementCompositionPreview.GetElementVisual(repositionTarget);
    Compositor compositor = targetVisual.Compositor;

    // Create an animation to animate targetVisual's Offset property to its
    final value
}
```

```

var repositionAnimation = compositor.CreateVector3KeyFrameAnimation();
repositionAnimation.Duration = TimeSpan.FromSeconds(0.66);
repositionAnimation.Target = "Offset";
repositionAnimation.InsertExpressionKeyFrame(1.0f, "this.FinalValue");

// Run this animation when the Offset Property is changed
var repositionAnimations =
compositor.CreateImplicitAnimationCollection();
repositionAnimations["Offset"] = repositionAnimation;

targetVisual.ImplicitAnimations = repositionAnimations;
}

```

Drop shadow

Apply a pixel-perfect drop shadow to a **UIElement**, for example an **Ellipse** containing a picture. Since the shadow requires a **SpriteVisual** created by the app, we need to create a "host" element which will contain the **SpriteVisual** using **ElementCompositionPreview.SetElementChildVisual**.

Implementation overview

1. Get the handout **Visual** for the host element
2. Create a **Windows.UI.Composition DropShadow**
3. Configure the **DropShadow** to get its shape from the target element via a mask
 - **DropShadow** is rectangular by default, so this is not necessary if the target is rectangular
4. Attach shadow to a new **SpriteVisual**, and set the **SpriteVisual** as the child of the host element
5. Bind size of the **SpriteVisual** to the size of the host using an **ExpressionAnimation**

XAML

```

<Grid Width="200" Height="200">
    <Canvas x:Name="ShadowHost" />
    <Ellipse x:Name="CircleImage">
        <Ellipse.Fill>
            <ImageBrush ImageSource="Assets/Images/2.jpg"
Stretch="UniformToFill" />
        </Ellipse.Fill>
    </Ellipse>
</Grid>

```

C#


```

public MainPage()
{
    InitializeComponent();
    InitializeDropShadow(ShadowHost, CircleImage);
}

private void InitializeDropShadow(UIElement shadowHost, Shape shadowTarget)
{
    Visual hostVisual =
    ElementCompositionPreview.GetElementVisual(shadowHost);
    Compositor compositor = hostVisual.Compositor;

    // Create a drop shadow
    var dropShadow = compositor.CreateDropShadow();
    dropShadow.Color = Color.FromArgb(255, 75, 75, 80);
    dropShadow.BlurRadius = 15.0f;
    dropShadow.Offset = new Vector3(2.5f, 2.5f, 0.0f);
    // Associate the shape of the shadow with the shape of the target
    element
    dropShadow.Mask = shadowTarget.GetAlphaMask();

    // Create a Visual to hold the shadow
    var shadowVisual = compositor.CreateSpriteVisual();
    shadowVisual.Shadow = dropShadow;

    // Add the shadow as a child of the host in the visual tree
    ElementCompositionPreview.SetElementChildVisual(shadowHost,
    shadowVisual);

    // Make sure size of shadow host and shadow visual always stay in sync
    var bindSizeAnimation =
    compositor.CreateExpressionAnimation("hostVisual.Size");
    bindSizeAnimation.SetReferenceParameter("hostVisual", hostVisual);

    shadowVisual.StartAnimation("Size", bindSizeAnimation);
}

```

The following two listings show the [C++/WinRT](#) and [C++/CX](#) equivalents of the previous C# code using the same XAML structure.

C++/WinRT

```

#include <winrt/Windows.UI.Composition.h>
#include <winrt/Windows.UI.Xaml.h>
#include <winrt/Windows.UI.Xaml.Hosting.h>
#include <winrt/Windows.UI.Xaml.Shapes.h>
...
MainPage()
{
    InitializeComponent();
    InitializeDropShadow(ShadowHost(), CircleImage());
}

```

```

int32_t MyProperty();
void MyProperty(int32_t value);

void InitializeDropShadow(Windows::UI::Xaml::UIElement const& shadowHost,
Windows::UI::Xaml::Shapes::Shape const& shadowTarget)
{
    auto hostVisual{
Windows::UI::Xaml::Hosting::ElementCompositionPreview::GetElementVisual(shad
owHost) };
    auto compositor{ hostVisual.Compositor() };

    // Create a drop shadow
    auto dropShadow{ compositor.CreateDropShadow() };
    dropShadow.Color(Windows::UI::ColorHelper::FromArgb(255, 75, 75, 80));
    dropShadow.BlurRadius(15.0f);
    dropShadow.Offset(Windows::Foundation::Numerics::float3{ 2.5f, 2.5f,
0.0f });
    // Associate the shape of the shadow with the shape of the target
element
    dropShadow.Mask(shadowTarget.GetAlphaMask());

    // Create a Visual to hold the shadow
    auto shadowVisual = compositor.CreateSpriteVisual();
    shadowVisual.Shadow(dropShadow);

    // Add the shadow as a child of the host in the visual tree

Windows::UI::Xaml::Hosting::ElementCompositionPreview::SetElementChildVisual
(shadowHost, shadowVisual);

    // Make sure size of shadow host and shadow visual always stay in sync
    auto bindSizeAnimation{
compositor.CreateExpressionAnimation(L"hostVisual.Size") };
    bindSizeAnimation.SetReferenceParameter(L"hostVisual", hostVisual);

    shadowVisual.StartAnimation(L"Size", bindSizeAnimation);
}

```

C++

```

#include "WindowsNumerics.h"

MainPage::MainPage()
{
    InitializeComponent();
    InitializeDropShadow(ShadowHost, CircleImage);
}

void MainPage::InitializeDropShadow(Windows::UI::Xaml::UIElement^
shadowHost, Windows::UI::Xaml::Shapes::Shape^ shadowTarget)
{
    auto hostVisual =

```

```

Windows::UI::Xaml::Hosting::ElementCompositionPreview::GetElementVisual(shadowHost);
    auto compositor = hostVisual->Compositor;

    // Create a drop shadow
    auto dropShadow = compositor->CreateDropShadow();
    dropShadow->Color = Windows::UI::ColorHelper::FromArgb(255, 75, 75, 80);
    dropShadow->BlurRadius = 15.0f;
    dropShadow->Offset = Windows::Foundation::Numerics::float3(2.5f, 2.5f, 0.0f);
    // Associate the shape of the shadow with the shape of the target element
    dropShadow->Mask = shadowTarget->GetAlphaMask();

    // Create a Visual to hold the shadow
    auto shadowVisual = compositor->CreateSpriteVisual();
    shadowVisual->Shadow = dropShadow;

    // Add the shadow as a child of the host in the visual tree

Windows::UI::Xaml::Hosting::ElementCompositionPreview::SetElementChildVisual(shadowHost, shadowVisual);

    // Make sure size of shadow host and shadow visual always stay in sync
    auto bindSizeAnimation = compositor->CreateExpressionAnimation("hostVisual.Size");
    bindSizeAnimation->SetReferenceParameter("hostVisual", hostVisual);

    shadowVisual->StartAnimation("Size", bindSizeAnimation);
}

```

Frosted glass

Create an effect that blurs and tints background content. Note that developers need to install the Win2D NuGet package to use effects. See the [Win2D homepage](#) for installation instructions.

Implementation overview

1. Get handout **Visual** for the host element
2. Create a blur effect tree using Win2D and **CompositionEffectSourceParameter**
3. Create a **CompositionEffectBrush** based on the effect tree
4. Set input of the **CompositionEffectBrush** to a **CompositionBackdropBrush**, which allows an effect to be applied to the content behind a **SpriteVisual**
5. Set the **CompositionEffectBrush** as the content of a new **SpriteVisual**, and set the **SpriteVisual** as the child of the host element. You could alternatively use a **XamlCompositionBrushBase**.

6. Bind size of the **SpriteVisual** to the size of the host using an **ExpressionAnimation**

XAML

```
<Grid Width="300" Height="300" Grid.Column="1">
    <Image
        Source="Assets/Images/2.jpg"
        Width="200"
        Height="200" />
    <Canvas
        x:Name="GlassHost"
        Width="150"
        Height="300"
        HorizontalAlignment="Right" />
</Grid>
```

C#

```
public MainPage()
{
    InitializeComponent();
    InitializeFrostedGlass(GlassHost);
}

private void InitializeFrostedGlass(UIElement glassHost)
{
    Visual hostVisual =
    ElementCompositionPreview.GetElementVisual(glassHost);
    Compositor compositor = hostVisual.Compositor;

    // Create a glass effect, requires Win2D NuGet package
    var glassEffect = new GaussianBlurEffect
    {
        BlurAmount = 15.0f,
        BorderMode = EffectBorderMode.Hard,
        Source = new ArithmeticCompositeEffect
        {
            MultiplyAmount = 0,
            Source1Amount = 0.5f,
            Source2Amount = 0.5f,
            Source1 = new CompositionEffectSourceParameter("backdropBrush"),
            Source2 = new ColorSourceEffect
            {
                Color = Color.FromArgb(255, 245, 245, 245)
            }
        }
    };

    // Create an instance of the effect and set its source to a
    CompositionBackdropBrush
    var effectFactory = compositor.CreateEffectFactory(glassEffect);
    var backdropBrush = compositor.CreateBackdropBrush();
    var effectBrush = effectFactory.CreateBrush();
```

```

effectBrush.SetSourceParameter("backdropBrush", backdropBrush);

// Create a Visual to contain the frosted glass effect
var glassVisual = compositor.CreateSpriteVisual();
glassVisual.Brush = effectBrush;

// Add the blur as a child of the host in the visual tree
ElementCompositionPreview.SetElementChildVisual(glassHost, glassVisual);

// Make sure size of glass host and glass visual always stay in sync
var bindSizeAnimation =
compositor.CreateExpressionAnimation("hostVisual.Size");
bindSizeAnimation.SetReferenceParameter("hostVisual", hostVisual);

glassVisual.StartAnimation("Size", bindSizeAnimation);
}

```

Additional Resources

- [Visual Layer overview](#)
- [ElementCompositionPreview](#) class
- Advanced UI and Composition samples in the [WindowsCompositionSamples](#) [GitHub](#) [↗](#)
- [BasicXamlInterop](#) sample [↗](#)
- [ParallaxingListItems](#) sample [↗](#)

XAML lighting

Article • 03/16/2023

[CompositionLight](#) objects are used in conjunction with [SceneLightingEffect](#) to simulate dynamic lighting and reflectivity.

You can apply lights to [Visuals](#) and XAML [UIElements](#).

Applying lights to XAML UIElements

[XamlLight](#) objects are used to apply [CompositionLights](#) to dynamically light XAML UIElements. XamlLight provides methods for targeting UIElements or XAML Brushes, applying lights to trees of UIElements, and helping manage the lifetime of CompositionLight resources based on whether they're currently in use.

- If you target a **Brush** with a XamlLight then the portions of any UIElements using that Brush are lit by the light.
- If you target a **UIElement** with a XamlLight then the entire UIElement and its child UIElements are all lit by the light.

Creating and using a XamlLight

[XamlLight](#) is a base class which can be used to create custom lights.

This example shows the definition for a custom XamlLight that applies a multicolored spotlight to targeted UIElements and Brushes.

C#

```
public sealed class OrangeSpotLight : XamlLight
{
    // Register an attached property that lets you set a UIElement
    // or Brush as a target for this light type in markup.
    public static readonly DependencyProperty IsTargetProperty =
        DependencyProperty.RegisterAttached(
            "IsTarget",
            typeof(bool),
            typeof(OrangeSpotLight),
            new PropertyMetadata(null, OnIsTargetChanged)
        );

    public static void SetIsTarget(DependencyObject target, bool value)
    {
        target.SetValue(IsTargetProperty, value);
    }
}
```

```

public static Boolean GetIsTarget(DependencyObject target)
{
    return (bool)target.GetValue(IsTargetProperty);
}

// Handle attached property changed to automatically target and untarget
// UIElements and Brushes.
private static void OnIsTargetChanged(DependencyObject obj,
DependencyPropertyChangedEventArgs e)
{
    var isAdding = (bool)e.NewValue;

    if (isAdding)
    {
        if (obj is UIElement)
        {
            XamlLight.AddTargetElement(GetIdStatic(), obj as UIElement);
        }
        else if (obj is Brush)
        {
            XamlLight.AddTargetBrush(GetIdStatic(), obj as Brush);
        }
    }
    else
    {
        if (obj is UIElement)
        {
            XamlLight.RemoveTargetElement(GetIdStatic(), obj as
UIElement);
        }
        else if (obj is Brush)
        {
            XamlLight.RemoveTargetBrush(GetIdStatic(), obj as Brush);
        }
    }
}

protected override void OnConnected(UIElement newElement)
{
    if (CompositionLight == null)
    {
        // OnConnected is called when the first target UIElement is
        // shown on the screen.
        // This lets you delay creation of the composition object until
        // it's actually needed.
        var spotLight = Window.Current.Compositor.CreateSpotLight();
        spotLight.InnerConeColor = Colors.Orange;
        spotLight.OuterConeColor = Colors.Yellow;
        spotLight.InnerConeAngleInDegrees = 30;
        spotLight.OuterConeAngleInDegrees = 45;
        CompositionLight = spotLight;
    }
}

```

```

protected override void OnDisconnected(UIElement oldElement)
{
    // OnDisconnected is called when there are no more target UIElements
    on the screen.
    // The CompositionLight should be disposed when no longer required.
    // For SDK 15063, see Remarks in the XamlLight class documentation.
    if (CompositionLight != null)
    {
        CompositionLight.Dispose();
        CompositionLight = null;
    }
}

protected override string GetId()
{
    return GetIdStatic();
}

private static string GetIdStatic()
{
    // This specifies the unique name of the light.
    // In most cases you should use the type's FullName.
    return typeof(OrangeSpotLight).FullName;
}
}

```

You can then apply this light to any XAML UIElement or Brush to light them. This example shows different potential usages.

❗ Important

For **C++/WinRT**, remove the two occurrences of

`local:OrangeSpotLight.IsTarget="True"` from the markup below. The attached properties are already set in code-behind.

XAML

```

<StackPanel Width="100">
    <StackPanel.Lights>
        <local:OrangeSpotLight/>
    </StackPanel.Lights>

    <!-- This border is lit by the OrangeSpotLight, but its content is not.
    -->
    <Border BorderThickness="4" Margin="2">
        <Border.BorderBrush>
            <SolidColorBrush x:Name="spotlitBrush" Color="White"
local:OrangeSpotLight.IsTarget="True"/>
        </Border.BorderBrush>
        <Rectangle Fill="LightGray" Height="20"/>
    </Border>
</StackPanel>

```



```

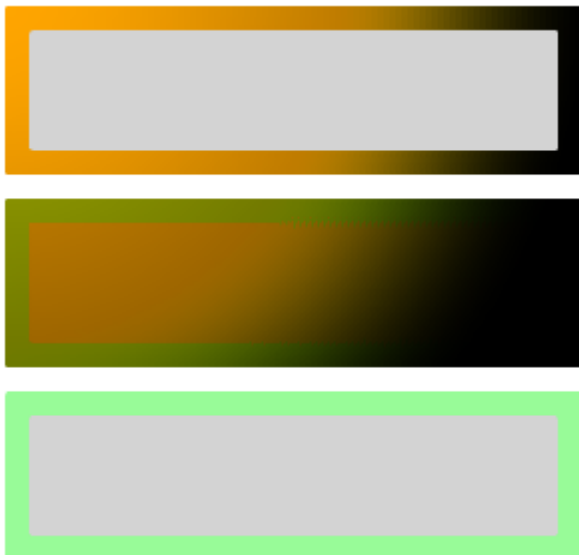
</Border>

<!-- This border and its content are lit by the OrangeSpotLight. -->
<Border x:Name="spotlitUIElement" BorderThickness="4"
BorderBrush="PaleGreen" Margin="2"
    local:OrangeSpotLight.IsTarget="True">
    <Rectangle Fill="LightGray" Height="20"/>
</Border>

<!-- This border and its content are not lit by the OrangeSpotLight. -->
<Border BorderThickness="4" BorderBrush="PaleGreen" Margin="2">
    <Rectangle Fill="LightGray" Height="20"/>
</Border>
</StackPanel>

```

The results of this XAML look like this.



Important

Setting `UIElement.Lights` in markup as shown in the above example is only supported for apps with a Minimum Version equal to the Windows 10 Creators Update or later. For apps that target earlier versions, lights must be created in code-behind.

Additional Resources

- Advanced UI and Composition samples in the [WindowsCompositionSamples GitHub](#).

Application development for Windows as a service

Article • 10/20/2022

Applies to

- Windows 11
- Windows 10
- Windows 10 Mobile
- Windows 10 IoT Core

In today's environment, where user expectations frequently are set by device-centric experiences, complete product cycles need to be measured in months, not years. Additionally, new releases must be made available on a continual basis, and must be deployable with minimal impact on users. Microsoft designed Windows 10 to meet these requirements by implementing a new approach to innovation, development, and delivery called [Windows as a service \(WaaS\)](#). The key to enabling significantly shorter product cycles while maintaining high quality levels is an innovative community-centric approach to testing that Microsoft has implemented for Windows 10. The community, known as Windows Insiders, is comprised of millions of users around the world. When Windows Insiders opt in to the community, they test many builds over the course of a product cycle and provide feedback to Microsoft through an iterative methodology called *flighting*.

Builds distributed as flights provide the Windows engineering team with significant data regarding how well builds are performing in actual use. *Flighting* with Windows Insiders also enables Microsoft to test builds in much more diverse hardware, application, and networking environments than in the past, and to identify issues far more quickly. As a result, Microsoft believes that community-focused *flighting* will enable both a faster pace of innovation delivery and better public release quality than ever.

Windows 10 and Windows 11 release types and cadences

Although Microsoft releases flight builds to Windows Insiders, Microsoft will publish two types of Windows 10 and Windows 11 releases broadly to the public on an ongoing basis:

Feature updates install the latest new features, experiences, and capabilities on devices that are already running Windows 10. Because feature updates contain an entire copy of Windows, they are also what customers use to install Windows 10 on existing devices running Windows 7 or Windows 8.1, and on new devices where no operating system is installed. Microsoft expects to publish updates semi-annually.

Quality updates deliver security issue resolutions and other important bug fixes. Quality updates will be provided to improve each feature currently in support, on a cadence of one or more times per month. Microsoft will continue publishing quality updates on Update Tuesday (sometimes referred to as Patch Tuesday). Additionally, Microsoft may publish additional quality updates for Windows 10 outside the Update Tuesday process when required to address customer needs.

During Windows 10 development, Microsoft streamlined the Windows product engineering and release cycle so that we can deliver the features, experiences, and functionality customers want, more quickly than ever. We also created new ways to deliver and install feature updates and quality updates that simplify deployments and on-going management, broaden the base of employees who can be kept current with the latest Windows capabilities and experiences, and lower total cost of ownership. Hence we have implemented new servicing options – referred to as Semi-Annual Channel, and Long-Term Servicing Channel (LTSC) – that provide pragmatic solutions to keep more devices more current in enterprise environments than was previously possible.

The following table shows describes the various servicing channels and their key attributes.

Servicing option	Availability of new feature upgrades for installation	Servicing lifetime	Key benefits	Supported editions
Semi-Annual Channel (Targeted)	Immediately after first published by Microsoft	18 months	Makes new features available to users as soon as possible	Home, Pro, Education, Enterprise, Mobile, IoT Core, Windows 10 IoT Core Pro (IoT Core Pro)
Semi-Annual Channel	Approximately 4 months after first published by Microsoft	18 months from when first published	Provides additional time to test new feature upgrades before deployment	Pro, Education, Enterprise, Mobile Enterprise, IoT Core Pro

Servicing option	Availability of new feature upgrades for installation	Servicing lifetime	Key benefits	Supported editions
Long-Term Servicing Channel (LTSC)	Immediately after published by Microsoft	10 Years	Enables long-term deployment of selected Windows 10 releases in low-change configurations	Enterprise LTSC

For more information, see [Windows 10/11 servicing options for updates and upgrades](#).

Supporting apps in Windows as a service

The traditional approach for supporting apps has been to release a new app version in response to a Windows release. This assumes that there are breaking changes in the underlying OS that could potentially cause a regression with the application. This model involves a dedicated development and validation cycle that requires our ISV partners to align with the Windows release cadence.

In the Windows as a service model, Microsoft is making a commitment to maintaining the compatibility of the underlying OS. This means Microsoft will make a concerted effort to ensure that there are no breaking changes that impact the app ecosystem negatively. In this scenario, when there is a release of a Windows build, most apps (those with no kernel dependencies) will continue to work.

In view of this change, Microsoft recommends that our ISV partners decouple their app release and support from specific Windows builds. Our mutual customers are better served by an application lifecycle approach. This means when an application version is released it will be supported for a certain period of time irrespective of however many Windows builds are released in the interim. The ISV makes a commitment to provide support for that specific version of the app as long as it is supported in the lifecycle. Microsoft follows a similar lifecycle approach for Windows that can be referenced [here](#).

This approach will reduce the burden of maintaining an app schedule that aligns with Windows releases. ISV partners should be free to release features or updates at their own cadence. We feel that our partners can keep their customer base updated with the latest app updates independent of a Windows release. In addition, our customers do not have to seek an explicit support statement whenever a Windows build is released. Here is an example of a support statement that covers how an app may be supported across different versions of the OS:

Example of an application lifecycle support statement

Contoso is a software development company and is the owner of the popular Mojave app which has a major share in the enterprise space. Contoso releases its next major release Mojave 14.0 and declares mainstream support for a period of three years from the release date. During mainstream support all updates and support are complimentary for the licensed product. Contoso also declares an additional two years of extended support where customers can purchase updates and support for a grace period. Beyond the extended support end date this product version is no longer supported. During the period of mainstream support Contoso will support Mojave 14.0 on all released builds of Windows. Contoso will also release updates to Mojave as necessary and independent of the Windows product releases.

In the following sections, you will find additional information about the steps Microsoft takes to maintain the compatibility of the underlying OS. You will also find guidance on steps you can take to help maintain the compatibility of the combined OS and app ecosystem. There is a section on how to leverage Windows flighting builds to detect app regressions before a Windows build is released. Lastly, we describe how we use an instrumentation and telemetry-driven approach to increase the quality of Windows builds. We recommend ISVs adopt a similar approach with their app portfolio.

Key changes since Windows 7 to ensure app compatibility

We understand that compatibility matters to developers. ISVs and developers want to ensure their apps will run as expected on all supported versions of the Windows OS. Consumers and businesses have a key investment here—they want to ensure that the apps they have paid for will continue to work. We know that compatibility is the primary criteria for purchase decisions. Apps that are well written based on best practices will lead to much less code churn when a new Windows version is released and will reduce fragmentation—these apps have a reduced engineering investment to maintain, and a faster time to market.

In the Windows 7 timeframe, compatibility was very much a reactive approach. In Windows 8, we started looking at this differently, working within Windows to ensure that compatibility was by design rather than an afterthought.

Windows 10 is the most compatible-by-design version of the OS to date. Here are some key ways we accomplished this:

- **App telemetry:** This helps us understand app popularity in the Windows ecosystem to inform compatibility testing.
- **ISV partnerships:** Work directly with external partners to provide them with data and help fix issues that our users experience.
- **Design reviews, upstream detection:** Partner with feature teams to reduce the number of breaking changes in Windows. Compatibility review is a gate that our feature teams must pass.
- **Communication:** Tighter control over API changes and improved communication.
- **Flighting and feedback loop:** Windows insiders receive flighted builds that help improve our ability to find compatibility issues before a final build is released to customers. This feedback process not only exposes bugs, but ensures we are shipping features our users want.

Best practices for app compatibility

Microsoft uses diagnostic and usage data to identify and troubleshoot problems, improve our products and services, and provide our users with personalized experiences. The usage data we collect also extends to the apps that PCs in the Windows ecosystem are running. Based on what our customers use, we build our list to test these apps, devices, and drivers against new versions of the Windows OS.

Windows 10 has been the most compatible version of Windows to-date, with over 90% compatibility against thousands of popular apps. The Windows Compatibility team commonly reaches out to our ISV partners to provide feedback if issues are discovered, so that we can partner together on solutions. Ideally, we'd like our common customers to be able to update Windows seamlessly and without losing functionality in either their OS or the apps they depend on for their productivity or entertainment.

The following sections contain some best practices Microsoft recommends so you can ensure your apps are compatible with Windows 10.

Windows version check

The OS version has been incremented with Windows 10. This means that the internal version number has been changed to 10.0. As in the past, we go to great lengths to maintain application and device compatibility after an OS version change. For most app categories (without any kernel dependencies), the change will not negatively impact app functionality, and existing apps will continue to work fine on Windows 10.

The manifestation of this change is app-specific. This means any app that specifically checks for the OS version will get a higher version number, which can lead to one or more of the following situations:

- App installers might not be able to install the app, and apps might not be able to start.
- Apps might become unstable or crash.
- Apps might generate error messages, but continue to function properly.

Some apps perform a version check and simply pass a warning to users. However, there are apps that are bound very tightly to a version check (in the drivers, or in kernel mode to avoid detection). In these cases, the app will fail if an incorrect version is found.

Rather than a version check, we recommend one of the following approaches:

- If the app is dependent on specific API functionality, ensure you target the correct API version.
- Ensure you detect the change via `APISet` or another public API, and do not use the version as a proxy for some feature or fix. If there are breaking changes and a proper check is not exposed, then that is a bug.
- Ensure the app does NOT check for version in odd ways, such as via the registry, file versions, offsets, kernel mode, drivers, or other means. If the app absolutely needs to check the version, use the `GetVersion` APIs, which should return the major, minor, and build number.
- If you are using the [GetVersion](#) API, remember that the behavior of this API has changed since Windows 8.1.

If you own apps such as antimalware or firewall apps, you should work through your usual feedback channels and via the Windows Insider program.

Undocumented APIs

Your apps should not call undocumented Windows APIs, or take dependency on specific Windows file exports or registry keys. This can lead to broken functionality, data loss, and potential security issues. If there is functionality your app requires that is not available, this is an opportunity to provide feedback through your usual feedback channels and via the Windows Insider program.

Develop Universal Windows Platform (UWP) and Centennial apps

We encourage all Win32 app ISVs to develop [Universal Windows Platform \(UWP\)](#) and, specifically, Centennial apps moving forward. There are great benefits to developing these app packages rather than using traditional Win32 installers. UWP apps are also supported in the [Microsoft Store](#), so it's easier for you to update your users to a consistent version automatically, lowering your support costs.

If your Win32 app types do not work with the Centennial model, we highly recommend that you use the right installer and ensure this is fully tested. An installer is your user or customer's first experience with your app, so ensure that this works well. All too often, this doesn't work well or it hasn't been fully tested for all scenarios. The [Windows App Certification Kit](#) can help you test the install and uninstall of your Win32 app and help you identify use of undocumented APIs, as well as other basic performance-related best-practice issues, before your users do.

Best practices:

- Use installers that work for both 32-bit and 64-bit versions of Windows.
- Design your installers to run on multiple scenarios (user or machine level).
- Keep all Windows redistributables in the original packaging – if you repackage these, it's possible that this will break the installer.
- Schedule development time for your installers—these are often overlooked as a deliverable during the software development lifecycle.

Optimized test strategies and flighting

Windows OS flighting refers to the interim builds available to Windows Insiders before a final build is released to the general population. The more Insiders that flight these interim builds, the more feedback we receive on the build quality, compatibility, etc., and this helps improve quality of the final builds. You can participate in this flighting program to ensure that your apps work as expected on iterative builds of the OS. We also encourage you to provide feedback on how these flighted builds are working for you, issues you run into, and so on.

If your app is in the Store, you can flight your app via the Store, which means that your app will be available for our Windows Insider population to install. Users can install your app and you can receive preliminary feedback on your app before you release it to the general population. The follow sections outline the steps for testing your apps against Windows flighted builds.

Step 1: Become a Windows Insider and participate in flighting

As a [Windows Insider](#), [↗](#) you can help shape the future of Windows—your feedback will help us improve features and functionality in the platform. This is a vibrant community where you can connect with other enthusiasts, join forums, trade advice, and learn about upcoming Insider-only events.

Since you'll have access to preview builds of Windows 10, Windows 10 Mobile, and the latest Windows SDK and Emulator, you'll have all the tools at your disposal to develop great apps and explore what's new in the Universal Windows Platform and the Microsoft Store.

This is also a great opportunity to build great hardware, with preview builds of the hardware development kits so you can develop universal drivers for Windows. The IoT Core Insider Preview is also available on supported IoT development boards, so you can build amazing connected solutions using the Universal Windows Platform.

Before you become a Windows Insider, please note that participation is intended for users who:

- Want to try out software that's still in development.
- Want to share feedback about the software and the platform.
- Don't mind lots of updates or a UI design that might change significantly over time.
- Really know their way around a PC and feel comfortable troubleshooting problems, backing up data, formatting a hard drive, installing an operating system from scratch, or restoring an old one if necessary.
- Know what an ISO file is and how to use it.
- Aren't installing it on their everyday computer or device.

Step 2: Test your scenarios

Once you have updated to a flighted build, the following are some sample test cases to help you get started on testing and gathering feedback. For most of these tests, ensure you cover both x86 and AMD64 systems. **Clean install test:** On a clean install of Windows 10, ensure your app is fully functional. If your app fails this test and the upgrade test, then it's likely that the issue is caused by underlying OS changes or bugs in the app. If after investigation, the former is the case, be sure to use the Windows Insider program to provide feedback and partner on solutions.

Upgrade Test: Check that your app works after upgrading from a down-level version of Windows (i.e. Windows 7 or Windows 8.1) to Windows 10. Your app shouldn't cause roll backs during upgrade, and should continue to work as expected after upgrade—this is crucial to achieve a seamless upgrade experience.

Reinstall Test: Ensure that app functionality can be restored by reinstalling your app after you upgrade the PC to Windows 10 from a down-level OS. If your app didn't pass the upgrade test and you have not been able to narrow down the cause of these issues,

it's possible that a reinstall can restore lost functionality. A passing reinstall test indicates that parts of the app may not have been migrated to Windows 10.

OS\Device Features Test: Ensure that your app works as expected if your app relies on specific functionality in the OS. Common areas for testing include the following, often against a selection of the commonly used PC models to ensure coverage:

- Audio
- USB device functionality (keyboard, mouse, memory stick, external hard disk, and so on)
- Bluetooth
- Graphics\display (multi-monitor, projection, screen rotation, and so on)
- Touch screen (orientation, on-screen keyboard, pen, gestures, and so on)
- Touchpad (left\right buttons, tap, scroll, and so on)
- Pen (single\double tap, press, hold, eraser, and so on)
- Print\Scan
- Sensors (accelerometer, fusion, and so on)
- Camera

Step 3: Provide feedback

Let us know how your app is performing against flighted builds. As you discover issues with your app during testing, please log bugs via the partner portal if you have access, or through your Microsoft representative. We encourage this information so that we can build a quality experience for our users together.

Related topics

[Windows 10/11 servicing options for updates and upgrades](#)

Windows Runtime components

Article • 05/09/2024

A Windows Runtime component is a self-contained software module that you can author, reference, and use with any Windows Runtime language (including C#, C++/WinRT, Visual Basic, JavaScript, and C++/CX). You can use Visual Studio to create a Windows Runtime component that can be consumed by either an app that uses the [Windows App SDK](#) or by a Universal Windows Platform (UWP) app.

ⓘ **Note**

For C++ developers, we recommend that you use [C++/WinRT](#) for new applications. C++/WinRT is an entirely standard modern C++17 language projection for Windows Runtime (WinRT) APIs, implemented as a header-file-based library, and designed to provide you with first-class access to the modern Windows API. To learn how to create a Windows Runtime component using C++/WinRT, see [Windows Runtime components with C++/WinRT](#).

ⓘ **Note**

For C# developers writing desktop apps in .NET 6 or later, use C#/WinRT to author a Windows Runtime component. See [Author Windows Runtime components with C#/WinRT](#).

 **Expand table**

Topic	Description
Windows Runtime components with C++/WinRT	This topic shows how to use C++/WinRT to create and consume a Windows Runtime component—a component that's callable from a Universal Windows app built using any Windows Runtime language.
Windows Runtime components with C++/CX	This topic shows how to use C++/CX to create a Windows Runtime component—a component that's callable from a Universal Windows app built using any Windows Runtime language.
Walkthrough of creating a C++/CX Windows Runtime component, and calling it from JavaScript or C#	This walkthrough shows how to create a basic Windows Runtime component DLL that's callable from JavaScript, C#, or Visual Basic. Before you begin this walkthrough, make sure that you understand concepts such as the Abstract Binary Interface (ABI), ref classes, and the Visual C++ Component Extensions that make working with ref classes easier. For

Topic	Description
	more information, see Creating Windows Runtime components in C++ and Visual C++ Language Reference (C++/CX) .
Windows Runtime components with C# and Visual Basic	You can use managed code to create your own Windows Runtime types, packaged in a Windows Runtime component. You can use your component in Universal Windows Platform (UWP) apps with C++, JavaScript, Visual Basic, or C#. This topic outlines the rules for creating a component, and discusses some aspects of .NET support for the Windows Runtime. In general, that support is designed to be transparent to the .NET programmer. However, when you create a component to use with JavaScript or C++, you need to be aware of differences in the way those languages support the Windows Runtime.
Walkthrough of creating a C# or Visual Basic Windows Runtime component, and calling it from JavaScript	This walkthrough shows how you can use .NET with Visual Basic or C# to create your own Windows Runtime types, packaged in a Windows Runtime component, and how to call the component from your Universal Windows app built for Windows using JavaScript.
Raising Events in Windows Runtime components	If your Windows Runtime component raises an event of a user-defined delegate type on a background thread (worker thread) and you want JavaScript to be able to receive the event, you can implement and/or raise it in one of these ways:

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Windows Runtime components with C++/WinRT

Article • 10/20/2022

This topic shows how to use [C++/WinRT](#) to create and consume a Windows Runtime component—a component that's callable from a Universal Windows app built using any Windows Runtime language.

There are several reasons for building a Windows Runtime component in C++/WinRT.

- To enjoy the performance advantage of C++ in complex or computationally intensive operations.
- To reuse standard C++ code that's already written and tested.
- To expose Win32 functionality to a Universal Windows Platform (UWP) app written in, for example, C#.

In general, when you author your C++/WinRT component, you can use types from the standard C++ library, and built-in types, except at the application binary interface (ABI) boundary where you're passing data to and from code in another `.winmd` package. At the ABI, use Windows Runtime types. In addition, in your C++/WinRT code, use types such as delegate and event to implement events that can be raised from your component and handled in another language. See [C++/WinRT](#) for more info about C++/WinRT.

The remainder of this topic walks you through how to author a Windows Runtime component in C++/WinRT, and then how to consume it from an application.

The Windows Runtime component that you'll build in this topic contains a runtime class representing a thermometer. The topic also demonstrates a Core App that consumes the thermometer runtime class, and calls a function to adjust the temperature.

ⓘ Note

For info about installing and using the [C++/WinRT Visual Studio Extension \(VSIX\)](#) and the NuGet package (which together provide project template and build support), see [Visual Studio support for C++/WinRT](#).

ⓘ Important

For essential concepts and terms that support your understanding of how to consume and author runtime classes with C++/WinRT, see [Consume APIs with C++/WinRT](#) and [Author APIs with C++/WinRT](#).

Naming best practice for Windows Runtime component dlls

Important

This section describes the naming convention that we recommend you use for the `.dll` file (DLL) that you build your Windows Runtime component into. It's all about the activation sequence that C++/WinRT follows when you consume a runtime class from a Windows Runtime component.

When activating a class factory, C++/WinRT first tries a call to [RoGetActivationFactory](#). If that fails, then C++/WinRT tries to find a DLL to load directly. Windows Runtime activation is always based on a fully-qualified class name. The logic is to remove the class name (from that fully-qualified class name), and then look for a DLL named for the full namespace that remains. If that's not found, remove the most-specific segment name, and repeat.

So for example if the class being activated has a fully-qualified name of **Contoso.Instruments.ThermometerWRC.Thermometer**, and [RoGetActivationFactory](#) fails, then we'll first look for a `Contoso.Instruments.ThermometerWRC.dll`. If that's not found, then we'll look for `Contoso.Instruments.dll`, and then for `Contoso.dll`.

When a DLL is found (in that sequence), we'll use that DLL's [DllGetActivationFactory](#) entry point to try to obtain the factory directly (rather than indirectly via the [RoGetActivationFactory](#) function that we first attempted). Even so, the end result is indistinguishable to the caller and to the DLL.

This process is completely automatic—no registration or tooling is needed. If you're authoring a Windows Runtime component, then you just need to use a naming convention for your DLLs that works with the process just described. And if you're consuming a Windows Runtime component and it's not named correctly, then you have the option to rename it as described.

Create a Windows Runtime component (ThermometerWRC)

Begin by creating a new project in Microsoft Visual Studio. Create a **Windows Runtime Component (C++/WinRT)** project, and name it *ThermometerWRC* (for "thermometer Windows Runtime component"). Make sure that **Place solution and project in the same directory** is unchecked. Target the latest generally-available (that is, not preview) version of the Windows SDK. Naming the project *ThermometerWRC* will give you the easiest experience with the rest of the steps in this topic.

Don't build the project yet.

The newly-created project contains a file named `Class.idl`. In Solution Explorer, rename that file `Thermometer.idl` (renaming the `.idl` file automatically renames the dependent `.h` and `.cpp` files, too). Replace the contents of `Thermometer.idl` with the listing below.

idl

```
// Thermometer.idl
namespace ThermometerWRC
{
    runtimeclass Thermometer
    {
        Thermometer();
        void AdjustTemperature(Single deltaFahrenheit);
    };
}
```

Save the file. The project won't build to completion at the moment, but building now is a useful thing to do because it generates the source code files in which you'll implement the **Thermometer** runtime class. So go ahead and build now (the build errors you can expect to see at this stage have to do with `Class.h` and `Class.g.h` not being found).

During the build process, the `midl.exe` tool is run to create your component's Windows Runtime metadata file (which is

`\ThermometerWRC\Debug\ThermometerWRC\ThermometerWRC.winmd`). Then, the `cppwinrt.exe` tool is run (with the `-component` option) to generate source code files to support you in authoring your component. These files include stubs to get you started implementing the **Thermometer** runtime class that you declared in your IDL. Those stubs are `\ThermometerWRC\ThermometerWRC\Generated Files\sources\Thermometer.h` and `Thermometer.cpp`.

Right-click the project node and click **Open Folder in File Explorer**. This opens the project folder in File Explorer. There, copy the stub files `Thermometer.h` and `Thermometer.cpp` from the folder `\ThermometerWRC\ThermometerWRC\Generated Files\sources\` and into the folder that contains your project files, which is `\ThermometerWRC\ThermometerWRC\`, and replace the files in the destination. Now, let's open `Thermometer.h` and `Thermometer.cpp` and implement our runtime class. In `Thermometer.h`, add a new private member to the implementation (*not* the factory implementation) of **Thermometer**.

C++/WinRT

```
// Thermometer.h
...
namespace winrt::ThermometerWRC::implementation
{
    struct Thermometer : ThermometerT<Thermometer>
    {
        ...

        private:
            float m_temperatureFahrenheit { 0.f };
    };
    ...
}
```

In `Thermometer.cpp`, implement the **AdjustTemperature** method as shown in the listing below.

C++/WinRT

```
// Thermometer.cpp
...
namespace winrt::ThermometerWRC::implementation
{
    void Thermometer::AdjustTemperature(float deltaFahrenheit)
    {
        m_temperatureFahrenheit += deltaFahrenheit;
    }
}
```

You'll see a `static_assert` at the top of `Thermometer.h` and `Thermometer.cpp`, which you'll need to remove. Now the project will build.

If any warnings prevent you from building, then either resolve them or set the project property **C/C++ > General > Treat Warnings As Errors** to **No (/WX-)**, and build the project again.

Create a Core App (ThermometerCoreApp) to test the Windows Runtime component

Now create a new project (either in your *ThermometerWRC* solution, or in a new one). Create a **Core App (C++/WinRT)** project, and name it *ThermometerCoreApp*. Set *ThermometerCoreApp* as the startup project if the two projects are in the same solution.

❗ Note

As mentioned earlier, the Windows Runtime metadata file for your Windows Runtime component (whose project you named *ThermometerWRC*) is created in the folder `\ThermometerWRC\Debug\ThermometerWRC\`. The first segment of that path is the name of the folder that contains your solution file; the next segment is the subdirectory of that named `Debug`; and the last segment is the subdirectory of that named for your Windows Runtime component. If you didn't name your project *ThermometerWRC*, then your metadata file will be in the folder `\<YourProjectName>\Debug\<YourProjectName>\`.

Now, in your Core App project (*ThermometerCoreApp*), add a reference, and browse to `\ThermometerWRC\Debug\ThermometerWRC\ThermometerWRC.winmd` (or add a project-to-project reference, if the two projects are in the same solution). Click **Add**, and then **OK**. Now build *ThermometerCoreApp*. In the unlikely event that you see an error that the payload file `readme.txt` doesn't exist, exclude that file from the Windows Runtime component project, rebuild it, then rebuild *ThermometerCoreApp*.

During the build process, the `cppwinrt.exe` tool is run to process the referenced `.winmd` file into source code files containing projected types to support you in consuming your component. The header for the projected types for your component's runtime classes—named `ThermometerWRC.h`—is generated into the folder `\ThermometerCoreApp\ThermometerCoreApp\Generated Files\winrt\`.

Include that header in `App.cpp`.

C++/WinRT

```
// App.cpp
...
#include <winrt/ThermometerWRC.h>
...
```

Also in `App.cpp`, add the following code to instantiate a **Thermometer** object (using the projected type's default constructor), and call a method on the thermometer object.

C++/WinRT

```
struct App : implements<App, IFrameworkViewSource, IFrameworkView>
{
    ThermometerWRC::Thermometer m_thermometer;
    ...

    void OnPointerPressed(IInspectable const &, PointerEventArgs const &
args)
    {
        m_thermometer.AdjustTemperature(1.f);
        ...
    }
    ...
};
```

Each time you click the window, you increment the thermometer object's temperature. You can set breakpoints if you want to step through the code to confirm that the application really is calling into the Windows Runtime component.

Next steps

To add even more functionality, or new Windows Runtime types, to your C++/WinRT Windows Runtime component, you can follow the same patterns shown above. First, use IDL to define the functionality you want to expose. Then build the project in Visual Studio to generate a stub implementation. And then complete the implementation as appropriate. Any methods, properties, and events that you define in IDL are visible to the application that consumes your Windows Runtime Component. For more info about IDL, see [Introduction to Microsoft Interface Definition Language 3.0](#).

For an example of how to add an event to your Windows Runtime Component, see [Author events in C++/WinRT](#).

Troubleshooting

Symptom	Remedy
---------	--------

Symptom	Remedy
In a C++/WinRT app, when consuming a C# Windows Runtime component that uses XAML, the compiler produces an error of the form "'MyNamespace_XamlTypeInfo': is not a member of 'winrt::MyNamespace'" — where <i>MyNamespace</i> is the name of the Windows Runtime component's namespace.	In <code>pch.h</code> in the consuming C++/WinRT app, add <code>#include <winrt/MyNamespace.MyNamespace_XamlTypeInfo.h></code> — replacing <i>MyNamespace</i> as appropriate.

Windows Runtime components with C++/CX

Article • 10/26/2022

ⓘ Note

This topic exists to help you maintain your C++/CX application. But we recommend that you use **C++/WinRT** for new applications. C++/WinRT is an entirely standard modern C++17 language projection for Windows Runtime (WinRT) APIs, implemented as a header-file-based library, and designed to provide you with first-class access to the modern Windows API. To learn how to create a Windows Runtime component using C++/WinRT, see [Windows Runtime components with C++/WinRT](#).

This topic shows how to use C++/CX to create a Windows Runtime component—a component that's callable from a Universal Windows app built using any Windows Runtime language (C#, Visual Basic, C++, or JavaScript).

There are several reasons for building a Windows Runtime component in C++.

- To get the performance advantage of C++ in complex or computationally intensive operations.
- To reuse code that's already written and tested.

When you build a solution that contains a JavaScript or .NET project, and a Windows Runtime component project, the JavaScript project files and the compiled DLL are merged into one package, which you can debug locally in the simulator or remotely on a tethered device. You can also distribute just the component project as an Extension SDK. For more information, see [Creating a Software Development Kit](#).

In general, when you code your C++/CX component, use the regular C++ library and built-in types, except at the abstract binary interface (ABI) boundary where you are passing data to and from code in another .winmd package. There, use Windows Runtime types and the special syntax that C++/CX supports for creating and manipulating those types. In addition, in your C++/CX code, use types such as delegate and event to implement events that can be raised from your component and handled in JavaScript, Visual Basic, C++, or C#. For more information about the C++/CX syntax, see [Visual C++ Language Reference \(C++/CX\)](#).

Casing and naming rules

JavaScript

JavaScript is case-sensitive. Therefore, you must follow these casing conventions:

- When you reference C++ namespaces and classes, use the same casing that's used on the C++ side.
- When you call methods, use camel casing even if the method name is capitalized on the C++ side. For example, a C++ method `GetDate()` must be called from JavaScript as `getDate()`.
- An activatable class name and namespace name can't contain UNICODE characters.

.NET

The .NET languages follow their normal casing rules.

Instantiating the object

Only Windows Runtime types can be passed across the ABI boundary. The compiler will raise an error if the component has a type like `std::wstring` as a return type or parameter in a public method. The Visual C++ component extensions (C++/CX) built-in types include the usual scalars such as `int` and `double`, and also their typedef equivalents `int32`, `float64`, and so on. For more information, see [Type System \(C++/CX\)](#).

C++

```
// ref class definition in C++
public ref class SampleRefClass sealed
{
    // Class members...

    // #include <valarray>
public:
    double LogCalc(double input)
    {
        // Use C++ standard library as usual.
        return std::log(input);
    }
};
```

JavaScript

```
//Instantiation in JavaScript (requires "Add reference > Project reference")  
var nativeObject = new CppComponent.SampleRefClass();
```

C#

```
//Call a method and display result in a XAML TextBlock  
var num = nativeObject.LogCalc(21.5);  
ResultText.Text = num.ToString();
```

C++/CX built-in types, library types, and Windows Runtime types

An activatable class (also known as a ref class) is one that can be instantiated from another language such as JavaScript, C# or Visual Basic. To be consumable from another language, a component must contain at least one activatable class.

A Windows Runtime component can contain multiple public activatable classes as well as additional classes that are known only internally to the component. Apply the [WebHostHidden](#) attribute to C++/CX types that are not intended to be visible to JavaScript.

All public classes must reside in the same root namespace which has the same name as the component metadata file. For example, a class that's named A.B.C.MyClass can be instantiated only if it's defined in a metadata file that's named A.winmd or A.B.winmd or A.B.C.winmd. The name of the DLL is not required to match the .winmd file name.

Client code creates an instance of the component by using the **new** (**New** in Visual Basic) keyword just as for any class.

An activatable class must be declared as **public ref class sealed**. The **ref class** keyword tells the compiler to create the class as a Windows Runtime compatible type, and the **sealed** keyword specifies that the class cannot be inherited. The Windows Runtime does not currently support a generalized inheritance model; a limited inheritance model supports creation of custom XAML controls. For more information, see [Ref classes and structs \(C++/CX\)](#).

For C++/CX, all the numeric primitives are defined in the default namespace. The [Platform](#) namespace contains C++/CX classes that are specific to the Windows Runtime type system. These include [Platform::String](#) class and [Platform::Object](#) class. The concrete collection types such as [Platform::Collections::Map](#) class and [Platform::Collections::Vector](#) class are defined in the [Platform::Collections](#) namespace. The public interfaces that these

types implement are defined in [Windows::Foundation::Collections Namespace \(C++/CX\)](#). It is these interface types that are consumed by JavaScript, C# and Visual Basic. For more information, see [Type System \(C++/CX\)](#).

Method that returns a value of built-in type

C++

```
// #include <valarray>
public:
    double LogCalc(double input)
    {
        // Use C++ standard library as usual.
        return std::log(input);
    }
```

JavaScript

```
//Call a method
var nativeObject = new CppComponent.SampleRefClass;
var num = nativeObject.logCalc(21.5);
document.getElementById('P2').innerHTML = num;
```

Method that returns a custom value struct

C++

```
namespace CppComponent
{
    // Custom struct
    public value struct PlayerData
    {
        Platform::String^ Name;
        int Number;
        double ScoringAverage;
    };

    public ref class Player sealed
    {
    private:
        PlayerData m_player;
    public:
        property PlayerData PlayerStats
        {
            PlayerData get(){ return m_player; }
            void set(PlayerData data) {m_player = data;}
        }
    }
```

```
};  
}
```

To pass user-defined value structs across the ABI, define a JavaScript object that has the same members as the value struct that's defined in C++/CX. You can then pass that object as an argument to a C++/CX method so that the object is implicitly converted to the C++/CX type.

JavaScript

```
// Get and set the value struct  
function GetAndSetPlayerData() {  
    // Create an object to pass to C++  
    var myData =  
        { name: "Bob Homer", number: 12, scoringAverage: .357 };  
    var nativeObject = new CppComponent.Player();  
    nativeObject.playerStats = myData;  
  
    // Retrieve C++ value struct into new JavaScript object  
    var myData2 = nativeObject.playerStats;  
    document.getElementById('P3').innerHTML = myData.name + " , " +  
    myData.number + " , " + myData.scoringAverage.toFixed(3);  
}
```

Another approach is to define a class that implements `IPropertySet` (not shown).

In the .NET languages, you just create a variable of the type that's defined in the C++/CX component.

C#

```
private void GetAndSetPlayerData()  
{  
    // Create a ref class  
    var player = new CppComponent.Player();  
  
    // Create a variable of a value struct  
    // type that is defined in C++  
    CppComponent.PlayerData myPlayer;  
    myPlayer.Name = "Babe Ruth";  
    myPlayer.Number = 12;  
    myPlayer.ScoringAverage = .398;  
  
    // Set the property  
    player.PlayerStats = myPlayer;  
  
    // Get the property and store it in a new variable  
    CppComponent.PlayerData myPlayer2 = player.PlayerStats;  
    ResultText.Text += myPlayer.Name + " , " + myPlayer.Number.ToString() +
```



```
        " , " + myPlayer.ScoringAverage.ToString());  
    }
```

Overloaded Methods

A C++/CX public ref class can contain overloaded methods, but JavaScript has limited ability to differentiate overloaded methods. For example, it can tell the difference between these signatures:

C++

```
public ref class NumberClass sealed  
{  
public:  
    int GetNumber(int i);  
    int GetNumber(int i, Platform::String^ str);  
    double GetNumber(int i, MyData^ d);  
};
```

But it can't tell the difference between these:

C++

```
int GetNumber(int i);  
double GetNumber(double d);
```

In ambiguous cases, you can ensure that JavaScript always calls a specific overload by applying the [Windows::Foundation::Metadata::DefaultOverload](#) attribute to the method signature in the header file.

This JavaScript always calls the attributed overload:

JavaScript

```
var nativeObject = new CppComponent.NumberClass();  
var num = nativeObject.getNumber(9);  
document.getElementById('P4').innerHTML = num;
```

.NET

The .NET languages recognize overloads in a C++/CX ref class just as in any .NET class.

DateTime

In the Windows Runtime, a [Windows::Foundation::DateTime](#) object is just a 64-bit signed integer that represents the number of 100-nanosecond intervals either before or after January 1, 1601. There are no methods on a `Windows.Foundation::DateTime` object. Instead, each language projects the `DateTime` in the way that is native to that language: the `Date` object in JavaScript and the `System.DateTime` and `System.DateTimeOffset` types in .NET.

C++

```
public ref class MyDateClass sealed
{
public:
    property Windows::Foundation::DateTime TimeStamp;
    void SetTime(Windows::Foundation::DateTime dt)
    {
        auto cal = ref new Windows::Globalization::Calendar();
        cal->SetDateTime(dt);
        TimeStamp = cal->GetDateTime(); // or TimeStamp = dt;
    }
};
```

When you pass a `DateTime` value from C++/CX to JavaScript, JavaScript accepts it as a `Date` object and displays it by default as a long-form date string.

JavaScript

```
function SetAndGetDate() {
    var nativeObject = new CppComponent.MyDateClass();

    var myDate = new Date(1956, 4, 21);
    nativeObject.setTime(myDate);

    var myDate2 = nativeObject.timeStamp;

    //prints long form date string
    document.getElementById('P5').innerHTML = myDate2;
}
```

When a .NET language passes a `System.DateTime` to a C++/CX component, the method accepts it as a `Windows::Foundation::DateTime`. When the component passes a `Windows::Foundation::DateTime` to a .NET method, the Framework method accepts it as a `DateTimeOffset`.

C#

```
private void DateTimeExample()
{
    // Pass a System.DateTime to a C++ method
    // that takes a Windows::Foundation::DateTime
    DateTime dt = DateTime.Now;
    var nativeObject = new CppComponent.MyDateClass();
    nativeObject.SetTime(dt);

    // Retrieve a Windows::Foundation::DateTime as a
    // System.DateTimeOffset
    DateTimeOffset myDate = nativeObject.TimeStamp;

    // Print the long-form date string
    ResultText.Text += myDate.ToString();
}
```

Collections and arrays

Collections are always passed across the ABI boundary as handles to Windows Runtime types such as `Windows::Foundation::Collections::IVector^` and `Windows::Foundation::Collections::IMap^`. For example, if you return a handle to a `Platform::Collections::Map`, it implicitly converts to a `Windows::Foundation::Collections::IMap^`. The collection interfaces are defined in a namespace that's separate from the C++/CX classes that provide the concrete implementations. JavaScript and .NET languages consume the interfaces. For more information, see [Collections \(C++/CX\)](#) and [Array and WriteOnlyArray \(C++/CX\)](#).

Passing IVector

C++

```
// Windows::Foundation::Collections::IVector across the ABI.
// #include <algorithm>
// #include <collection.h>
Windows::Foundation::Collections::IVector<int>^
SortVector(Windows::Foundation::Collections::IVector<int>^ vec)
{
    std::sort(begin(vec), end(vec));
    return vec;
}
```

JavaScript

```
var nativeObject = new CppComponent.CollectionExample();
// Call the method to sort an integer array
```

```

var inVector = [14, 12, 45, 89, 23];
var outVector = nativeObject.sortVector(inVector);
var result = "Sorted vector to array:";
for (var i = 0; i < outVector.length; i++)
{
    outVector[i];
    result += outVector[i].toString() + ",";
}
document.getElementById('P6').innerHTML = result;

```

The .NET languages see `IVector<T>` as `IList<T>`.

C#

```

private void SortListItems()
{
    IList<int> myList = new List<int>();
    myList.Add(5);
    myList.Add(9);
    myList.Add(17);
    myList.Add(2);

    var nativeObject = new CppComponent.CollectionExample();
    IList<int> mySortedList = nativeObject.SortVector(myList);

    foreach (var item in mySortedList)
    {
        ResultText.Text += " " + item.ToString();
    }
}

```

Passing IMap

C++

```

// #include <map>
// #include <collection.h>
Windows::Foundation::Collections::IMap<int, Platform::String^> ^GetMap(void)
{
    Windows::Foundation::Collections::IMap<int, Platform::String^> ^ret =
        ref new Platform::Collections::Map<int, Platform::String^>;
    ret->Insert(1, "One ");
    ret->Insert(2, "Two ");
    ret->Insert(3, "Three ");
    ret->Insert(4, "Four ");
    ret->Insert(5, "Five ");
    return ret;
}

```

JavaScript

```
// Call the method to get the map
var outputMap = nativeObject.getMap();
var mStr = "Map result:" + outputMap.lookup(1) + outputMap.lookup(2)
    + outputMap.lookup(3) + outputMap.lookup(4) + outputMap.lookup(5);
document.getElementById('P7').innerHTML = mStr;
```

The .NET languages see IMap and IDictionary<K, V>.

C#

```
private void GetDictionary()
{
    var nativeObject = new CppComponent.CollectionExample();
    IDictionary<int, string> d = nativeObject.GetMap();
    ResultText.Text += d[2].ToString();
}
```

Properties

A public ref class in C++/CX component extensions exposes public data members as properties, by using the property keyword. The concept is identical to .NET properties. A trivial property resembles a data member because its functionality is implicit. A non-trivial property has explicit get and set accessors and a named private variable that's the "backing store" for the value. In this example, the private member variable `_propertyAValue` is the backing store for `PropertyA`. A property can fire an event when its value changes, and a client app can register to receive that event.

C++

```
//Properties
public delegate void PropertyChangedHandler(Platform::Object^ sender, int
arg);
public ref class PropertyExample sealed
{
public:
    PropertyExample(){}

    // Event that is fired when PropertyA changes
    event PropertyChangedHandler^ PropertyChangedEvent;

    // Property that has custom setter/getter
    property int PropertyA
    {
        int get() { return m_propertyAValue; }
        void set(int propertyAValue)
```